

HSBI — Bielefeld University of Applied Sciences and Arts

Quantitative Research Methods

Summer Semester 2026 · Examiner: Prof. Dr. Christoph Weisser

Final Mock Examination (Lectures 1–12)

Comprehensive coverage of ISLP Chapters 2–8, 10 and 13,
with emphasis on Chapters 7 (splines/GAMs), 8 (trees), 10 (deep learning) and 13 (multiple testing)

— Version with detailed solutions —

Duration: 120 minutes

Total credit: 120 points

Allowed aids: non-programmable calculator; one handwritten A4 sheet of notes (both sides)

Name: _____

Matriculation no.: _____

Problem	P1	P2	P3	P4	P5	P6	P7	Total
Maximum points	16	8	20	16	20	20	20	120
Achieved points								

Instructions

- Justify all answers. Numerical results without a derivation or a brief reasoning receive no credit.
- Round final numerical results to 3 decimal places unless stated otherwise. Small deviations caused by carrying rounded intermediate results are accepted.
- All numerical constants you need (powers of e , logarithms, $(0.95)^{20}$, t -quantiles) are provided in the problems.
- The number of points per problem roughly equals the recommended working time in minutes.

Problem 1: Moving beyond linearity — splines

(16 points)

Lecture slides: Chapter 7 — Moving Beyond Linearity (Lecture 9).

- (a) [4 P] A *cubic spline* is a piecewise cubic polynomial that is continuous and has continuous first and second derivatives at each knot. How many degrees of freedom (free parameters, counting the intercept) does a cubic spline with K interior knots use? Justify your answer by counting the parameters of the polynomial pieces and the constraints at the knots, and evaluate the result for $K = 4$.
- (b) [4 P] How many degrees of freedom does a *natural* cubic spline with the same K knots use (again evaluate for $K = 4$)? Which additional constraint does a natural spline impose, in which region of the predictor space does it act, and why is this constraint useful in practice?
- (c) [4 P] A *smoothing spline* is the function g that minimises

$$\sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int g''(t)^2 dt.$$

Explain the role of the two terms, and describe the behaviour of the solution $\hat{g}$ in the two limiting cases $\lambda \rightarrow 0$ and $\lambda \rightarrow \infty$.

- (d) [4 P] Write down the *truncated-power basis* representation of a cubic spline with a single knot at ξ , define the truncated-power basis function explicitly, and explain in one sentence why adding this basis function does not destroy the smoothness of the fit at ξ .

Solution

(a) Degrees of freedom of a cubic spline. K interior knots split the range of X into $K + 1$ intervals, each carrying a cubic polynomial with 4 coefficients: $4(K + 1) = 4K + 4$ parameters. At each knot, continuity of g , g' and g'' imposes 3 constraints, i.e. $3K$ constraints in total. Hence

$$\text{df} = 4(K + 1) - 3K = K + 4.$$

For $K = 4$: $\text{df} = 4 + 4 = 8$ free parameters (equivalently: intercept, x , x^2 , x^3 plus one truncated-power basis function per knot $= 4 + K$).

(b) Natural cubic spline. A natural spline additionally requires the function to be *linear beyond the boundary knots*, i.e. in the two outer regions $X < \xi_1$ and $X > \xi_K$. Forcing the quadratic and cubic terms to vanish there removes 2 parameters per boundary region, i.e. 4 in total:

$$\text{df} = (K + 4) - 4 = K, \quad K = 4 : \text{df} = 4.$$

Usefulness: polynomials and unconstrained splines are notoriously erratic near the boundaries, where data are sparse; the linearity constraint stabilises the fit there (*lower variance* of predictions at and beyond the boundary), at the price of a small amount of additional bias.

(c) Smoothing spline. The first term (RSS) rewards fidelity to the data; the second term penalises roughness, since g'' measures the curvature (wiggleness) of g , and $\lambda \geq 0$ is the tuning parameter that trades the two off. $\lambda \rightarrow 0$: the penalty disappears; $\hat{g}$ can be made arbitrarily wiggly and *interpolates* the training observations exactly (zero training RSS, severe overfitting; effective $\text{df} \rightarrow n$). $\lambda \rightarrow \infty$: any curvature is infinitely expensive, so $g'' \equiv 0$ is enforced; $\hat{g}$ becomes a straight line and equals the *least-squares regression line* (effective $\text{df} \rightarrow 2$). Between the extremes, λ (chosen e.g. by LOOCV) controls the effective degrees of freedom, which decrease smoothly from n to 2.

(d) Truncated-power basis with one knot ξ .

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x - \xi)_+^3, \quad (x - \xi)_+^3 = \begin{cases} (x - \xi)^3, & x > \xi \\ 0, & x \leq \xi \end{cases}$$

The function $(x - \xi)_+^3$ and its first and second derivatives are all 0 at $x = \xi$, so adding it changes only the third derivative at the knot; the fit remains continuous with continuous first and second derivatives — exactly the defining smoothness of a cubic spline.

Problem 2: Generalised additive models

(8 points)

Lecture slides: Chapter 7 — Moving Beyond Linearity (Lecture 9).

- (a) [3 P] Write down the general form of a *generalised additive model* (GAM) for a quantitative response Y and predictors $X_1, \dots, X_p$, and explain in one sentence how it extends multiple linear regression.
- (b) [3 P] State one important *advantage* of GAMs that concerns the interpretation of the fitted

functions f_j , and the main structural *limitation* of GAMs — including how this limitation can be repaired.

- (c) [2 P] Describe in one or two sentences how the *backfitting* algorithm fits a GAM.

Solution

(a) Model form.

$$Y = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p) + \varepsilon.$$

Multiple linear regression is the special case $f_j(X_j) = \beta_j X_j$; a GAM replaces each linear component by a smooth non-linear function f_j (e.g. a natural or smoothing spline, or local regression), while keeping the *additive* structure.

(b) Advantage and limitation. *Advantage (interpretability):* because the model is additive, the contribution of each X_j to Y is captured by its own function f_j , which can be examined and plotted individually while the other variables are held fixed — non-linear effects remain as easy to read as in a coefficient table, one panel per predictor. *Limitation:* additivity excludes *interactions*: the effect of X_j cannot depend on the value of X_k . *Repair:* add interaction terms manually, e.g. a product term $X_j \times X_k$ or a low-dimensional smooth surface $f_{jk}(X_j, X_k)$ as an extra component.

(c) Backfitting. Cycle through the predictors and repeatedly re-fit each f_j (by a smoother, e.g. a spline) to the *partial residuals* $r_i = y_i - \beta_0 - \sum_{k \neq j} f_k(x_{ik})$, holding all other fitted functions fixed; iterate until the f_j stop changing.

Problem 3: Regression and classification trees by hand

(20 points)

Lecture slides: Chapter 8 — Tree-Based Methods (Lecture 10).

An online platform records for $n = 6$ products the advertising budget x (in 1,000 EUR) and the weekly sales y (in 1,000 units):

Product i	1	2	3	4	5	6
Budget x_i	1	2	3	4	5	6
Sales y_i	2	4	3	3	8	10

A regression tree with a *single* split at cutpoint s partitions the data into $R_1 = \{i : x_i < s\}$ and $R_2 = \{i : x_i \geq s\}$ and predicts the region mean $\hat{y}_{R_m}$ in each region.

- (a) [8 P] For each of the two candidate cutpoints $s = 2.5$ and $s = 4.5$, determine the two regions, the region means and the resulting total residual sum of squares $\text{RSS}(s) = \sum_{m=1}^2 \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2$. Which cutpoint does recursive binary splitting choose, and why?
- (b) [2 P] Using the split chosen in part (a), namely $s = 4.5$ (so $R_1 = \{i : x_i < 4.5\}$ with mean $\hat{y}_{R_1} = 3$ and $R_2 = \{i : x_i \geq 4.5\}$ with mean $\hat{y}_{R_2} = 9$), predict the weekly sales for a new product with advertising budget $x = 5$.
- (c) [6 P] A *classification* tree for a different task has a terminal node containing 8 training observations: 6 of class “Yes” and 2 of class “No”. Compute the class proportions, the Gini index $G = \sum_k \hat{p}_k(1 - \hat{p}_k)$ and the entropy $D = -\sum_k \hat{p}_k \ln \hat{p}_k$ of this node (use $\ln 0.75 = -0.2877$ and $\ln 0.25 = -1.3863$). Which class does the node predict?

- (d) [4 P] In *cost-complexity pruning* one minimises, for a subtree $T \subseteq T_0$ of the large tree T_0 ,

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|.$$

Explain the role of the tuning parameter α : what happens for $\alpha = 0$, what happens as α increases, and how is α chosen in practice?

Solution

(a) **Evaluating the two cutpoints.** Root node for reference: $\bar{y} = 30/6 = 5$.

Cutpoint $s = 2.5$: $R_1 = \{1, 2\}$ with y -values $\{2, 4\}$, mean $\hat{y}_{R_1} = 3$; $R_2 = \{3, 4, 5, 6\}$ with y -values $\{3, 3, 8, 10\}$, mean $\hat{y}_{R_2} = 24/4 = 6$.

$$\text{RSS}_{R_1} = (2-3)^2 + (4-3)^2 = 2, \quad \text{RSS}_{R_2} = (3-6)^2 + (3-6)^2 + (8-6)^2 + (10-6)^2 = 9+9+4+16 = 38,$$

so $\text{RSS}(2.5) = 2 + 38 = 40$.

Cutpoint $s = 4.5$: $R_1 = \{1, 2, 3, 4\}$ with y -values $\{2, 4, 3, 3\}$, mean $\hat{y}_{R_1} = 12/4 = 3$; $R_2 = \{5, 6\}$ with y -values $\{8, 10\}$, mean $\hat{y}_{R_2} = 9$.

$$\text{RSS}_{R_1} = 1 + 1 + 0 + 0 = 2, \quad \text{RSS}_{R_2} = (8-9)^2 + (10-9)^2 = 2,$$

so $\text{RSS}(4.5) = 2 + 2 = 4$.

Since $4 < 40$, recursive binary splitting is *greedy* and picks the cutpoint with the smallest RSS: $s = 4.5$. (It separates the four low-sales products, mean 3, from the two high-sales products, mean 9.)

(b) **Prediction for $x = 5$.** $x = 5 \geq 4.5$, so the new product falls into R_2 and the tree predicts $\hat{y} = \hat{y}_{R_2} = 9$ (i.e. 9,000 units per week).

(c) **Node impurity.** $\hat{p}_{\text{Yes}} = 6/8 = 0.75$, $\hat{p}_{\text{No}} = 2/8 = 0.25$.

$$G = 0.75 \cdot 0.25 + 0.25 \cdot 0.75 = 0.1875 + 0.1875 = \mathbf{0.375}.$$

$$D = -(0.75 \ln 0.75 + 0.25 \ln 0.25) = -(0.75 \cdot (-0.2877) + 0.25 \cdot (-1.3863)) = 0.2158 + 0.3466 = \mathbf{0.562}.$$

The node predicts the majority class, **Yes** (with estimated probability 0.75). Both G and D are small when the node is pure; here the node is moderately impure.

(d) **Role of α .** α is the price per terminal node: it trades off the fit of the subtree (first term) against its complexity $|T|$. For $\alpha = 0$ the criterion equals the training RSS and the full tree T_0 is optimal. As α increases, additional leaves must “pay for themselves”; branches whose RSS reduction is smaller than α are pruned, so the optimal subtree becomes smaller — one obtains a *nested sequence* of subtrees as α grows (weakest-link pruning). In practice α is selected by *cross-validation* (choose the α with the smallest CV error, then refit on all data), which balances bias against variance instead of optimising training fit.

Problem 4: Ensembles — bagging, random forests, boosting

(16 points)

Lecture slides: Chapter 8 — Tree-Based Methods (Lecture 10).

- (a) [6 P] Let $\hat{f}_1(x), \dots, \hat{f}_B(x)$ be B identically distributed (but not independent) tree predictions at a fixed point x , each with variance σ^2 and with pairwise correlation ρ . The variance of the bagged average is

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)\right) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2.$$

- (i) Explain briefly why this formula shows that bagging reduces variance, and which term limits the reduction. (ii) Evaluate the variance for $B = 100$, $\rho = 0.6$ and $\sigma^2 = 4$, and compare it with the variance of a single tree. (iii) What value does the variance approach as $B \rightarrow \infty$?
- (b) [4 P] How do *random forests* modify bagging at each split, and what is the recommended subset size m for classification as a function of p (evaluate for $p = 25$)? Explain — with reference to the ensemble-variance formula $\text{Var} = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ from part (a) — why this modification improves on bagging.
- (c) [4 P] A data scientist tunes a *boosting* model for regression trees with the following `scikit-learn` call:
- ```
GradientBoostingRegressor(n_estimators=5000,
 learning_rate=0.01, max_depth=1)
```
- These three constructor arguments are exactly the three tuning parameters of boosting for regression trees. State in one sentence each what `n_estimators`, `learning_rate` and `max_depth` control. What is the “slow learning” idea behind boosting, and how is it reflected in the values chosen here?
- (d) [2 P] What does the *out-of-bag* (OOB) error of a bagged model estimate, and why is it available essentially for free?

### Solution

**(a) Variance of the bagged average.** (i) The second term  $\frac{1-\rho}{B}\sigma^2$  is driven to 0 by averaging over many trees ( $B$  large), so averaging removes the “independent part” of the trees’ variability — this is the variance reduction of bagging. The first term  $\rho\sigma^2$  does *not* depend on  $B$ : because the bootstrap trees are grown on overlapping samples of the same data (and share the same strong predictors), they are positively correlated, and this correlated part of the variance cannot be averaged away. It limits the reduction. (ii) For  $B = 100$ ,  $\rho = 0.6$ ,  $\sigma^2 = 4$ :

$$\text{Var} = 0.6 \cdot 4 + \frac{1 - 0.6}{100} \cdot 4 = 2.4 + \frac{0.4 \cdot 4}{100} = 2.4 + 0.016 = \mathbf{2.416},$$

compared with  $\sigma^2 = 4$  for a single tree — a reduction of about 40%. (iii) As  $B \rightarrow \infty$  the variance approaches the floor  $\rho\sigma^2 = 0.6 \cdot 4 = \mathbf{2.4}$ . (Increasing  $B$  further does not overfit; it only stabilises the average.)

**(b) Random forests.** At each split, only a *random subset* of  $m$  of the  $p$  predictors is considered as splitting candidates; a fresh subset is drawn at every split. Recommended for classification:  $m \approx \sqrt{p}$ , so for  $p = 25$ :  $m = \sqrt{25} = \mathbf{5}$ . Why it helps: with bagging, a dominant predictor is used in the top split of almost every tree, so the trees look alike and  $\rho$  is high. Excluding it from  $\approx (p - m)/p$  of the split searches *decorrelates* the trees, i.e. lowers  $\rho$  — and by (a) this lowers the irreducible floor  $\rho\sigma^2$  of the ensemble variance, which mere averaging cannot touch.

**(c) Boosting — the three tuning parameters (constructor arguments).**

- `n_estimators` =  $B$ , the *number of trees*: how many small trees are added sequentially; unlike bagging, boosting *can* overfit if  $B$  is too large, so it is chosen by cross-validation (here  $B = 5000$ ).
- `learning_rate` =  $\lambda$ , the *shrinkage* (e.g. 0.01 or 0.001): scales the contribution of each new tree and thus the speed of learning; smaller  $\lambda$  usually needs larger  $B$  (here  $\lambda = 0.01$ ).
- `max_depth` =  $d$ , the *number of splits per tree* (interaction depth): the complexity of each tree and the interaction order of the model; often even  $d = 1$  (stumps, additive model) works well (here `max_depth` = 1).

*Slow learning*: each tree is fitted to the current *residuals*, and only a small fraction  $\lambda$  of it is added to the model; the model thus improves slowly and exactly in the regions where it

still performs poorly, instead of fitting the data hard in one step — which tends to generalise better. The chosen values encode exactly this: a *small* `learning_rate=0.01` with *many* trees (`n_estimators=5000`) and shallow `max_depth=1` stumps.

(d) **OOB error.** Each bootstrap sample leaves out on average about  $1/3$  of the observations (“out-of-bag”). Predicting each observation only with the trees that did *not* use it, and aggregating these predictions, yields the OOB error — a valid estimate of the *test error* of the bagged model. It is free because it reuses the bootstrap side product; no separate validation set or cross-validation loop is required.

## Problem 5: Neural networks by hand

(20 points)

Lecture slides: Chapter 10 — Deep Learning (Lecture 11).

Consider a tiny feed-forward network for a quantitative response with input  $x = (x_1, x_2) = (1, 2)$ , one hidden layer with two ReLU units ( $g(z) = \max\{0, z\}$ ) and a linear output unit:

$$A_1 = g(-1 + 1x_1 + 2x_2), \quad A_2 = g(-3 - 2x_1 + 1x_2), \quad f(x) = 1 + 2A_1 + 4A_2.$$

- [8 P]** Perform the forward pass step by step: compute both pre-activations, both activations  $A_1, A_2$  and the network output  $f(x)$ . Comment briefly on what the ReLU did to the second unit and why such non-linear activations are essential.
- [6 P]** A classification network with  $K = 3$  classes produces the logits (output-layer pre-activations)  $z = (z_1, z_2, z_3) = (1.0, 0.0, -1.0)$  for one observation whose true class is class 1. Compute all three softmax probabilities and the cross-entropy loss  $-\ln \hat{p}_{\text{true}}$ . (Use  $e^1 = 2.718$ ,  $e^0 = 1.000$ ,  $e^{-1} = 0.368$  and  $\ln 4.086 = 1.408$ .)
- [4 P]** Count the parameters of a fully connected network with  $p = 4$  inputs, one hidden layer with  $k = 3$  units and an output layer with  $K = 2$  units, where every hidden and output unit has a bias. Show the count separately for the two weight layers.
- [2 P]** A convolutional layer receives a  $32 \times 32$  image and uses filters of size  $F = 5$  with padding  $P = 2$  and stride  $S = 1$ . Compute the spatial size of the output feature map from  $(W - F + 2P)/S + 1$ .

## Solution

(a) **Forward pass.** Pre-activations:

$$z_1 = -1 + 1 \cdot 1 + 2 \cdot 2 = -1 + 1 + 4 = 4, \quad z_2 = -3 - 2 \cdot 1 + 1 \cdot 2 = -3 - 2 + 2 = -3.$$

ReLU activations:

$$A_1 = g(4) = \max\{0, 4\} = 4, \quad A_2 = g(-3) = \max\{0, -3\} = 0.$$

Output:

$$f(x) = 1 + 2 \cdot 4 + 4 \cdot 0 = 1 + 8 + 0 = \mathbf{9}.$$

The ReLU *switched the second unit off*: its negative pre-activation is clipped to 0, so unit 2 contributes nothing for this input (but would for other inputs). This thresholding is the source of non-linearity: without a non-linear  $g$ , the network would collapse into a single linear function of  $x$ , no matter how many layers it has.

(b) **Softmax and cross-entropy.** Exponentials:  $e^{1.0} = 2.718$ ,  $e^{0.0} = 1.000$ ,  $e^{-1.0} = 0.368$ ; their sum is  $2.718 + 1.000 + 0.368 = 4.086$ . Softmax probabilities:

$$\hat{p}_1 = \frac{2.718}{4.086} = 0.665, \quad \hat{p}_2 = \frac{1.000}{4.086} = 0.245, \quad \hat{p}_3 = \frac{0.368}{4.086} = 0.090,$$

which sum to 1.000. Cross-entropy loss for true class 1:

$$-\ln \hat{p}_1 = -\ln \frac{e^1}{4.086} = -(1 - \ln 4.086) = 1.408 - 1 = \mathbf{0.408}.$$

(Equivalently  $-\ln 0.665 \approx 0.408$ ; a perfect prediction  $\hat{p}_1 = 1$  would give loss 0.)

(c) **Parameter count for  $4 \rightarrow 3 \rightarrow 2$ .** Input  $\rightarrow$  hidden:  $4 \cdot 3 = 12$  weights + 3 biases = 15. Hidden  $\rightarrow$  output:  $3 \cdot 2 = 6$  weights + 2 biases = 8. Total:  $15 + 8 = \mathbf{23}$  parameters.

(d) **Convolution output size.**

$$\frac{W - F + 2P}{S} + 1 = \frac{32 - 5 + 2 \cdot 2}{1} + 1 = \frac{31}{1} + 1 = 32,$$

so the output feature map is  $\mathbf{32 \times 32}$  — padding  $P = 2$  with  $F = 5$ ,  $S = 1$  is exactly the “same” convolution that preserves the spatial size.

## Problem 6: Multiple testing

(20 points)

Lecture slides: Chapter 13 — Multiple Testing (Lecture 12).

- (a) [4 P] A lab performs  $m = 20$  independent hypothesis tests, each at level  $\alpha = 0.05$ , and suppose all 20 null hypotheses are true. Define the family-wise error rate (FWER) and compute it for this situation (use  $(0.95)^{20} = 0.3585$ ). Interpret the result in one sentence.
- (b) [7 P] A research group screens  $m = 6$  candidate biomarkers; the ordered  $p$ -values are

| $j$       | 1     | 2     | 3     | 4     | 5     | 6     |
|-----------|-------|-------|-------|-------|-------|-------|
| $p_{(j)}$ | 0.001 | 0.004 | 0.011 | 0.030 | 0.045 | 0.230 |

Control the FWER at  $\alpha = 0.05$  (i) with *Bonferroni* and (ii) with the *Holm step-down* procedure. Give the relevant thresholds, state for each method exactly which hypotheses are rejected, and explain why Holm can never reject fewer hypotheses than Bonferroni.

- (c) [6 P] Apply the *Benjamini–Hochberg* (BH) procedure with FDR level  $q = 0.05$  to the same six ordered  $p$ -values from part (b), namely  $p_{(1)}, \dots, p_{(6)} = 0.001, 0.004, 0.011, 0.030, 0.045, 0.230$ : give the thresholds  $jq/m$ , determine which hypotheses are rejected, and state precisely what “FDR control at  $q = 0.05$ ” guarantees for the resulting set of rejections.
- (d) [3 P] A genomics group screens 20,000 genes in an *exploratory* study; the most promising hits will be re-tested in an independent follow-up experiment. Should they control the FWER or the FDR? Justify your recommendation.

## Solution

(a) **FWER for  $m = 20$  independent tests.**  $\text{FWER} = \Pr(\text{at least one Type I error}) = \Pr(V \geq 1)$ , where  $V$  is the number of true nulls that are (falsely) rejected. With independence and all nulls true:

$$\text{FWER} = 1 - \Pr(\text{no false rejection}) = 1 - (1 - 0.05)^{20} = 1 - (0.95)^{20} = 1 - 0.3585 = \mathbf{0.6415}.$$

Interpretation: even though each single test is performed at the 5% level, there is a 64% chance of at least one false discovery among the 20 tests — testing many hypotheses without adjustment almost guarantees spurious findings.

**(b) Bonferroni and Holm at  $\alpha = 0.05$ .** *Bonferroni:* reject  $H_{(j)}$  iff  $p_{(j)} \leq \alpha/m = 0.05/6 = 0.00833$ .  $p_{(1)} = 0.001 \leq 0.00833$  ✓,  $p_{(2)} = 0.004 \leq 0.00833$  ✓,  $p_{(3)} = 0.011 > 0.00833$  ×.  $\Rightarrow$  Bonferroni rejects  $H_{(1)}$  and  $H_{(2)}$ : **2 rejections**.

*Holm:* compare  $p_{(j)}$  with  $\alpha/(m - j + 1)$  step by step and stop at the first failure:

| $j$                  | 1       | 2      | 3      | 4      | 5      | 6      |
|----------------------|---------|--------|--------|--------|--------|--------|
| $p_{(j)}$            | 0.001   | 0.004  | 0.011  | 0.030  | 0.045  | 0.230  |
| $\alpha/(m - j + 1)$ | 0.00833 | 0.0100 | 0.0125 | 0.0167 | 0.0250 | 0.0500 |
| reject?              | ✓       | ✓      | ✓      | stop   | —      | —      |

$0.001 \leq 0.00833$ ,  $0.004 \leq 0.0100$ ,  $0.011 \leq 0.0125$ , but  $0.030 > 0.0167$ : stop.  $\Rightarrow$  Holm rejects  $H_{(1)}, H_{(2)}, H_{(3)}$ : **3 rejections**.

*Why Holm  $\supseteq$  Bonferroni:* Holm's thresholds  $\alpha/(m - j + 1)$  are  $\geq \alpha/m$  for every  $j$  (with equality only at  $j = 1$ ), so every  $p$ -value that passes Bonferroni also passes Holm's less stringent later hurdles — Holm is uniformly more powerful, yet still controls the FWER at  $\alpha$  without any independence assumption.

**(c) Benjamini–Hochberg at  $q = 0.05$ .** Thresholds  $jq/m = j \cdot 0.05/6$ :

| $j$                   | 1       | 2      | 3      | 4          | 5      | 6      |
|-----------------------|---------|--------|--------|------------|--------|--------|
| $p_{(j)}$             | 0.001   | 0.004  | 0.011  | 0.030      | 0.045  | 0.230  |
| $jq/m$                | 0.00833 | 0.0167 | 0.0250 | 0.0333     | 0.0417 | 0.0500 |
| $p_{(j)} \leq jq/m$ ? | yes     | yes    | yes    | <b>yes</b> | no     | no     |

$L =$  largest  $j$  with  $p_{(j)} \leq jq/m$ :  $j = 4$  (since  $0.030 \leq 0.0333$ , while  $0.045 > 0.0417$  and  $0.230 > 0.0500$ ). BH rejects *all*  $H_{(j)}$  with  $j \leq L$ :  $H_{(1)}, \dots, H_{(4)}$ : **4 rejections**. Guarantee:  $\text{FDR} = E[V/\max(R, 1)] \leq q = 0.05$  — *on average* at most 5% of the rejected hypotheses are false discoveries. It does *not* guarantee that at most 5% of these particular 4 rejections are false, nor does it bound the probability of making any false rejection (that would be FWER control).

**(d) Exploratory genomics screening.** Control the **FDR**. With  $m = 20,000$  tests, FWER control (e.g. Bonferroni at  $0.05/20,000 = 2.5 \cdot 10^{-6}$ ) guards against even a single false positive and therefore has extremely low power — almost no true signals would survive. In an exploratory screen the cost of a few false leads is low, because every hit is validated in an independent follow-up experiment; what matters is that the *list* of discoveries is mostly reliable. FDR control at, say,  $q = 0.05$ – $0.2$  delivers exactly that: a rich list of candidates of which on average at most a known fraction is false.

## Problem 7: Cumulative essentials (Chapters 2–6)

(20 points)

Lecture slides: Chapters 2–6 (Lectures 1–8).

- (a) [4 P] True or false — justify briefly: “A more flexible statistical learning method always yields a lower *test* error than a less flexible one.”
- (b) [4 P] A bank fits a logistic regression for loan default in `statsmodels`,  $\log\left(\frac{p(x)}{1-p(x)}\right) = \beta_0 + \beta_1 \text{missed\_payments}$ , where `missed_payments` is the number of missed payments  $x$ . The call `sm.Logit(default, X).fit()` produces:

|                              | coef    | std err | $z$    | $P >  z $ |
|------------------------------|---------|---------|--------|-----------|
| <code>const</code>           | −3.5000 | 0.420   | −8.333 | 0.000     |
| <code>missed_payments</code> | 0.7000  | 0.150   | 4.667  | 0.000     |

Using the estimated coefficient of `missed_payments`, interpret the effect of *one additional missed payment* on the *odds* of default (use  $e^{0.7} = 2.014$ ). Why can the effect on the *probability* of default not be stated as a single number?

- (c) **[4 P]** Give one reason to prefer  $k$ -fold cross-validation with  $k = 5$  or  $10$  over *LOOCV*, and one reason to prefer it over a *single validation split*.
- (d) **[2 P]** You have  $p = 2,000$  predictors but expect only a handful to be truly related to the response. Ridge or lasso — which penalty is more appropriate, and why?
- (e) **[6 P]** Match each scenario to exactly one method from  $\{\text{linear regression, lasso, logistic regression, random forest}\}$  (each method used exactly once) and justify each choice in one sentence:
- (i) Estimate how much an additional square metre adds to the price of an apartment, with a confidence interval, from  $n = 2,000$  sales.
  - (ii) Predict a continuous drug response from  $p = 5,000$  gene-expression measurements for  $n = 100$  patients; clinicians want a short list of relevant genes.
  - (iii) Predict whether a loan applicant will default (yes/no) and report to the regulator how each applicant characteristic changes the odds of default.
  - (iv) Predict machine failures from hundreds of sensor variables with strong non-linear effects and interactions; only predictive accuracy matters.

### Solution

**(a) False.** Expected test error = variance + bias<sup>2</sup> + irreducible error. More flexibility reduces bias but *increases* variance; once the variance increase outweighs the bias reduction, the test error rises again — the characteristic U-shape. So a more flexible method wins only up to the sweet spot; e.g. when the true  $f$  is nearly linear or  $n$  is small, a flexible method overfits and a simple linear fit has *lower* test error. (Only the *training* error always decreases with flexibility.)

**(b) Odds ratio.** The `coef` column gives  $\hat{\beta}_1 = 0.7$  for `missed_payments` (highly significant,  $z = 4.667$ ). One additional missed payment increases the log-odds by  $\hat{\beta}_1 = 0.7$ , i.e. it *multiplies the odds* of default by  $e^{0.7} = 2.014$  — the odds roughly double, regardless of the starting value of  $x$ . The effect on the *probability* depends on where the observation sits on the S-shaped logistic curve: the same change in log-odds shifts  $p(x)$  a lot near  $p \approx 0.5$  but very little when  $p$  is close to 0 or 1 — hence no single number describes it.

**(c)  $k$ -fold CV with  $k = 5$  or  $10$ .** *vs. LOOCV:* the  $n$  LOOCV fits are trained on almost identical data sets, so their errors are highly positively correlated; averaging strongly correlated quantities reduces variance little, so the LOOCV estimate has *higher variance* than 5- or 10-fold CV (it is also computationally expensive:  $n$  fits instead of  $k$ , absent shortcut formulas). *vs. a single validation split:* the validation estimate depends strongly on the random split and overestimates the test error because only part of the data is used for training (*bias*);  $k$ -fold CV trains on  $(k - 1)/k$  of the data and averages over  $k$  folds, giving a less biased and more stable estimate.

**(d) Lasso.** The  $\ell_1$  penalty sets many coefficients *exactly to zero* and thus performs variable selection — ideal when the truth is sparse (a handful of signals among 2,000 predictors), and it yields an interpretable small model. Ridge shrinks coefficients but keeps all  $p = 2,000$  predictors in the model and tends to lose when most true coefficients are zero.

**(e) Matching.**

- (i) **Linear regression:** quantitative response, goal is *inference* (effect size with confidence interval) — exactly what OLS with standard errors provides.
- (ii) **Lasso:**  $p \gg n$  makes OLS infeasible; the  $\ell_1$  penalty regularises and selects a short list of genes.
- (iii) **Logistic regression:** binary response with interpretable coefficients ( $e^{\hat{\beta}_j}$  = odds ratios)

- suitable for regulatory reporting.
- (iv) **Random forest:** captures non-linearities and interactions automatically, is accurate and robust with many mixed predictors; interpretability is not required.

---

*End of the examination — good luck!*